

私域运营目标驱动 Agent 系统讲解

版本日期：2026-07-07。本文档基于当前本地项目 `/Users/wangjie/Desktop/mahjong_agent_core` 的实际代码整理，重点解释“消息如何进入系统、模型如何理解目标、上下文如何构建、工具如何执行、状态如何落库、如何审计回放”。

1. 系统定位

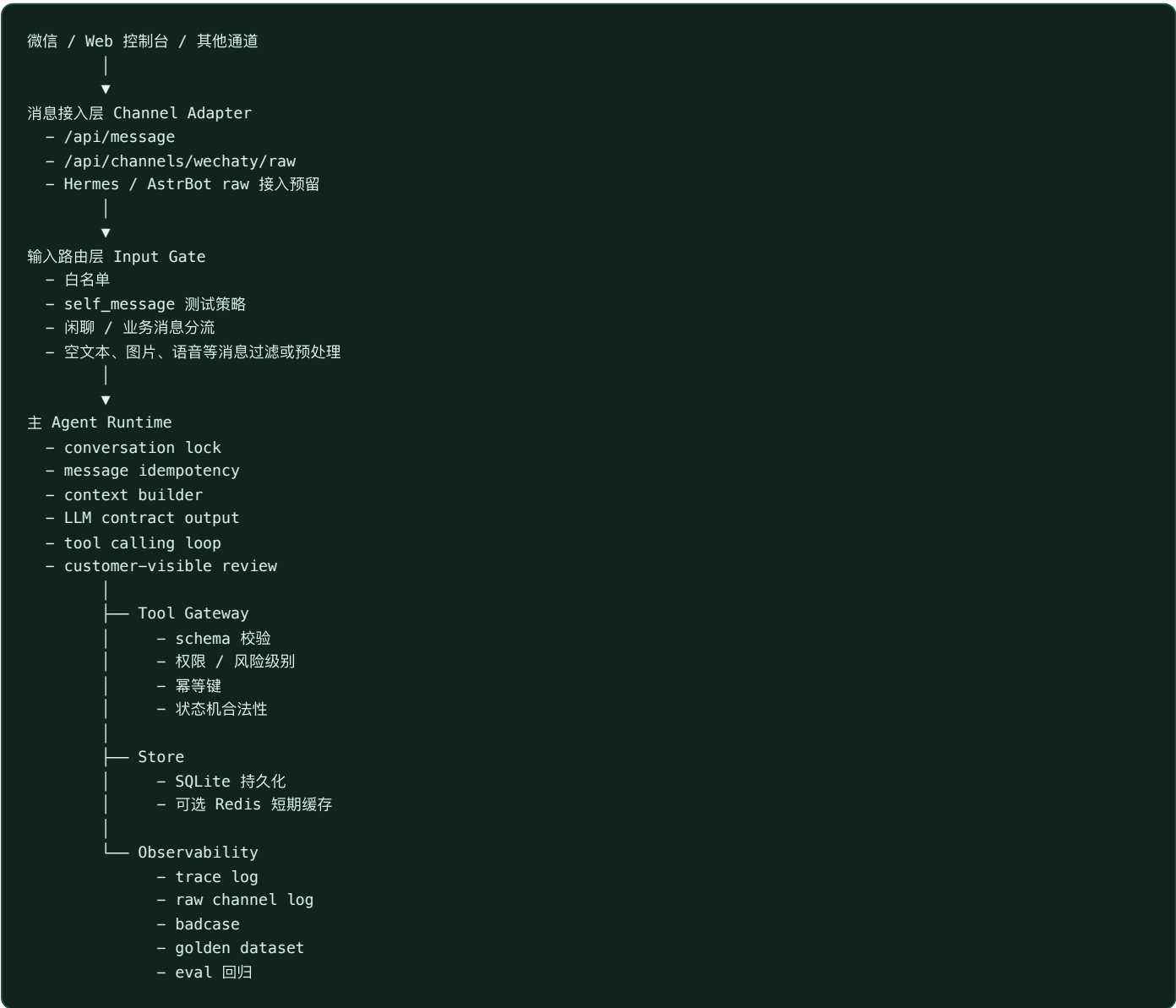
这个系统面向本地生活私域运营撮合场景。当前以棋牌室组局为主业务验证场景，但技术抽象是“私域消息输入 + 目标驱动 Agent + 后端受控工具 + 状态推进 + 审计回放”。

系统目标不是简单问答，而是让 Agent 接到用户消息后，能够基于上下文、客户画像、当前局池和工具结果，自主判断下一步：查现有局、建局、找候选人、生成邀约、等待用户、记录反馈或转人工。

当前能力边界

- 支持本地 Web 控制台输入，也支持 WeChaty 原始微信消息接入。
- 支持好友和群聊消息采集、白名单路由、自发消息测试、闲聊和业务消息分流。
- 支持多轮对话、上下文打包、Checkpoint 摘要、短期任务记忆和候选长期画像。
- 支持由模型决定工具调用和调用顺序，后端只负责契约校验、权限、幂等、状态机和落库。
- 所有客户可见文本都要经过审查流程，避免暴露系统、模型、内部工具、其他客户隐私和不应透露的执行细节。
- 当前微信测试阶段支持白名单用户；真实自动外发仍需要外发开关和人工审批策略控制。

2. 总体架构



核心设计原则

原则	含义	代码落点
模型做决策	模型负责理解用户目标、决定是否调用工具、决定工具调用顺序，并基于工具结果调整下一步。	src/mahjong_agent_runtime/runtime.py
后端做边界	后端不把业务语义写成无穷 if-else，只做工具契约、权限、幂等、并发顺序、状态机合法性和落库。	src/mahjong_agent_runtime/tools.py
客户可见文本过审	回复用户或候选人的文本必须经过审查，避免泄露系统内部、模型、其他用户隐私和未确认承诺。	CUSTOMER_VISIBLE_CONTENT_REVIEW_TOOL_NAME
可回放	每次输入、上下文、模型输出、工具调用、状态变更都写入 trace。	JsonlTraceRecorder

3. 入口与运行方式

服务入口

HTTP 服务入口在 `/Users/wangjie/Desktop/mahjong_agent_core/scripts/agent_runtime_app.py` 。启动后默认监听 `127.0.0.1:8790` 。

关键初始化链路

1. `build_runtime()` 读取环境变量和配置。
2. 创建 `SQLiteAgentStore` 作为主存储。
3. 创建 `ToolGateway` ，注册后端工具。
4. 创建 `AgentContextBuilder` ，负责构建模型上下文。
5. 创建 `AgentRuntime` ，运行主 Agent Loop。
6. 可选创建 `ContextSummaryManager` ，在对话变长后生成摘要 checkpoint。

重要环境变量

环境变量	默认值	作用
<code>MAHJONG_AGENT_PORT</code>	<code>8790</code>	本地服务端口。
<code>MAHJONG_AGENT_DB_PATH</code>	<code>data/agent_runtime.sqlite3</code>	SQLite 数据库路径。
<code>MAHJONG_AGENT_TRACE_PATH</code>	<code>logs/agent_runtime_trace.log</code>	主链路 trace 日志。
<code>MAHJONG_WECHATY_RAW_LOG_PATH</code>	<code>logs/wechaty_weixin_raw.jsonl</code>	WeChaty 原始消息日志。
<code>MAHJONG_AGENT_MAX_STEPS</code>	<code>8</code>	单轮最多 Agent loop 步数。
<code>MAHJONG_AGENT_MAX_CALLS_PER_TURN</code>	<code>8</code>	单轮最多模型调用次数。
<code>MAHJONG_AGENT_MAX_TOKENS_PER_CALL</code>	<code>24000</code>	单次模型调用预算上限。
<code>MAHJONG_AGENT_LLM_TIMEOUT_SECONDS</code>	<code>45</code>	主模型超时时间。

4. 消息接入

Web 控制台

Web 控制台调用 `/api/message` 。请求体会被转换成 `UserMessage` ：

```
{
  "conversation_id": "runtime_trial",
  "sender_id": "zhang",
  "sender_name": "张哥",
  "text": "五点0.5, 173, 无烟",
  "message_id": "可选, 外部消息源 ID"
}
```

控制台出现的默认 `runtime_trial` 、 `zhang` 、 `张哥` 是为了本地测试可用。生产通道必须由外部消息源传入真实的 `conversation` 、 `sender` 和 `message id` 。

WeChaty 原始消息

微信通道入口是 `/api/channels/wechaty/raw`。它先把 WeChaty 原始 payload 写入 `logs/wechaty_weixin_raw.jsonl`，再尝试转换成统一的 `UserMessage`。

```
{
  "channel": "wechaty",
  "conversation_id": "wechaty:contact:@xxx 或 wechaty:room:@xxx",
  "sender_id": "@contact-id",
  "sender_name": "微信昵称或备注",
  "source_message_id": "WeChaty message id",
  "raw_text": "用户文本",
  "is_room": false,
  "self_message": false,
  "payload": { "...": "完整原始消息" }
}
```

为什么要保留 raw log

- 调试通道问题：比如消息为空、message type 不支持、self_message 判断异常。
- 追溯真实微信消息：主 Agent 只看抽象后的 `UserMessage`，raw log 保留证据。
- 后续支持图片、语音、引用消息时，可以从 raw payload 反推字段结构。

5. 路由与意图识别

不是每条消息都会进入主 Agent

微信消息进入后，会先走输入门禁。它负责判断消息是否应该进入主业务 Agent，还是走闲聊流程，或者直接忽略。

阶段	职责	结果
通道白名单	只处理允许测试的联系人、群聊或 self_message。	不在白名单时不进入主流程。
输入门禁	判断消息是组局/查局/反馈等业务意图，还是普通闲聊。	业务消息进入主 Agent；闲聊走 casual chat。
主 Agent 语义理解	结合上下文、画像、当前局、短期记忆推断目标和下一步动作。	输出结构化 action 和 tool calls。

输入门禁提示词

WeChaty 输入门禁提示词在：

```
/Users/wangjie/Desktop/mahjong_agent_core/src/mahjong_agent_runtime/prompts/wechaty_input_gate.md
```

它是一个轻量判断任务，不负责完整组局，只决定是否路由到主 Agent。这样可以避免大量闲聊浪费主 Agent token，也减少误触发。

业务意图不是靠正则写死

主链路里，意图识别由模型基于上下文完成。后端会提供结构化上下文和工具列表，模型输出目标状态和工具调用。比如用户说：

```
“老板，今天下午，有没有打麻将的，0.5 或者 1 都行，烟也都可”
```

模型应该理解为“咨询现有局或希望撮合”，并优先调用 `search_current_games`，而不是后端用固定 if-else 把每种说法枚举出来。

6. 主 Agent Loop

主循环在 `/Users/wangjie/Desktop/mahjong_agent_core/src/mahjong_agent_runtime/runtime.py`。

处理流程

```
handle_user_message(message)
├─ conversation lock
├─ message idempotency check
├─ advance conversation version
├─ supersede pending outputs
└─
▼
_handle_once(...)
├─ append user turn
├─ build context
├─ call LLM
├─ parse AgentAction contract
├─
├─ if needs_tool:
│   └─ review customer-visible tool text if needed
│   └─ execute tools via ToolGateway
│   └─ append tool results
│   └─ loop again with previous_tool_results
├─
└─ if waiting_user/completed/needs_human:
    └─ review final reply
    └─ append assistant turn
    └─ return final output
```

为什么这是 Agent Loop

因为模型不是只做一次分类。它会在每一轮看到工具结果，然后决定下一步继续调用什么工具，直到可以回复用户或需要人工。典型例子：

1. 用户问“现在有没有 0.5 无烟人齐开的”。
2. 模型调用 `search_current_games`。
3. 工具返回没有匹配局。
4. 模型决定回复“现在没有，要组吗”，并等待用户。
5. 用户回复“组”。
6. 模型结合上一轮上下文，可能调用 `create_game` 和 `search_customers`。

7. 上下文构建

上下文构建器在 `/Users/wangjie/Desktop/mahjong_agent_core/src/mahjong_agent_runtime/context.py` 的 `AgentContextBuilder.build()`。

喂给模型的消息结构

```
[
  {
    "role": "system",
    "content": "agent_runtime_system.md 的完整系统提示词"
  },
  {
    "role": "user",
    "content": "{...结构化 JSON payload...}"
  }
]
```

payload 包含什么

字段	说明
runtime	运行时名称。
trace_id	本轮链路 ID，用于日志串联。
conversation_state	会话 ID、版本号、run id，用于处理并发和过期输出。
current_message	当前用户消息。
quoted_message_context	引用消息上下文预留，用于处理“用户引用前文回复”的场景。
recent_conversation	近期多轮对话，经过 token 预算打包。
conversation_checkpoint	长对话摘要，避免完整历史塞满上下文。
sender_profile	当前发送人的长期客户画像。
sender_relationships	当前发送人与其他客户的关系，如打过、不跟谁打。
task_memories	当前任务内立即生效的短期约束，比如“这局不和 C 打”。
pending_memory_candidates	候选长期画像，供模型参考但不当权威事实。
active_games	当前会话相关活跃局。
active_game_visible_summaries	面向客户可说的局摘要，减少模型乱改局况。
active_parties	当前活跃组局关系。
outbound_message_drafts	待审批或待发送的外发草稿。
available_tools	本轮允许模型调用的工具 schema。
previous_tool_results	上一轮 loop 工具返回结果。
planning_contract	目标规划约束。
output_contract	模型输出 JSON 契约。

近期对话如何打包

`ContextPackingPolicy.pack_turns()` 会从最近的 turn 往前扫描，把 turn 逐个加入上下文，直到接近 `max_recent_conversation_tokens` 。被省略的旧 turn 依赖 checkpoint 摘要承接。

当前 token 估算函数是近似实现：

```
def estimate_tokens(value):
    text = value if isinstance(value, str) else json.dumps(value, ensure_ascii=False, sort_keys=True)
    return max(1, len(text) // 4)
```

也就是说，它不是模型 tokenizer 的精确 token 计数，而是工程上可用的预算估算。优点是简单、快、不依赖特定模型；缺点是中文、JSON、英文混合时会有误差，所以真实生产可以替换成模型 tokenizer 或服务商返回的 usage 统计。

8. 摘要与 Checkpoint

摘要管理器在 `/Users/wangjie/Desktop/mahjong_agent_core/src/mahjong_agent_runtime/summary.py`。

什么时候生成摘要

每次主 Agent 完成一轮用户消息处理后，会调用 `maybe_summarize_after_turn()`。它不会每轮都摘要，而是满足以下条件才触发：

条件	默认值	原因
会话 turn 数达到阈值	<code>MAHJONG_CONTEXT_SUMMARY_MIN_TURNS=12</code>	太短的会话没有摘要价值。
距离上次摘要有足够新 turn	<code>MAHJONG_CONTEXT_SUMMARY_MIN_TURNS_SINCE_LAST=6</code>	避免每轮重复摘要。
近期对话 token 估算超过阈值	<code>MAHJONG_CONTEXT_SUMMARY_TOKEN_THRESHOLD=3000</code>	只有上下文开始变重时才压缩。
摘要输入预算未超限	<code>MAHJONG_CONTEXT_SUMMARY_MAX_INPUT_TOKENS=6000</code>	避免摘要本身也超预算。
模型摘要置信度达标	<code>MAHJONG_CONTEXT_SUMMARY_MIN_CONFIDENCE=0.6</code>	低置信摘要不能覆盖 checkpoint。

Checkpoint 存什么

Checkpoint 存在 `runtime_conversation_checkpoints` 表中，核心内容是：

- `summary`：会话摘要。
- `facts`：结构化事实，例如当前目标、关键偏好、已确认信息。
- `open_questions`：还没确认的问题。
- `source_trace_id`：摘要来自哪次 trace，方便回溯。

为什么摘要不是长期画像

摘要是会话级记忆，用来续接多轮上下文；长期画像是用户级稳定事实。比如“这局不和 C 打”应先进入 `task_memories`，不是直接写长期画像。只有用户明确表达稳定偏好，或者会话结束后模型提炼且置信度足够高，才进入长期画像候选。

9. 提示词设计

主 Agent 系统提示词

主提示词文件：

`/Users/wangjie/Desktop/mahjong_agent_core/src/mahjong_agent_runtime/prompts/agent_runtime_system.md`

提示词的核心要求

- 你是私域运营撮合助手，目标是帮助用户完成组局或查局。
- 不要暴露自己是模型、Agent、系统、工具或内部流程。
- 不要把槽位字段机械复述给客户，回复要像老板说话。
- 不要把工具执行细节、候选人数量、内部评分、画像推断过程告诉客户。
- 如果信息足够，要主动调用工具推进，而不是只说“我帮你看看”。
- 如果用户只是问现有局，要先查局；没有现成局再问是否要组。
- 如果用户明确要组局，要创建局或查候选人，不能停在口头承诺。

- 当前任务约束必须写入 `record_user_memory`，例如“不和 C 打”“只接受无烟”“最多四小时”。
- 候选长期画像不能直接改数据库，只能作为 pending candidate 记录，等待后续提炼或审核。

辅助提示词

文件	用途
<code>wechaty_input_gate.md</code>	判断微信消息是否进入主 Agent。
<code>wechaty_casual_chat_reply.md</code>	闲聊回复生成。
<code>customer_visible_text_generation.md</code>	客户可见话术生成。
<code>agent_runtime_reply_self_review.md</code>	回复自审。
<code>context_summary.md</code>	上下文摘要生成。

10. 模型输出契约

模型不能自由输出一段文本。它必须返回 JSON object，后端用契约解析成 `AgentAction`。

核心字段

```
{
  "goal": "本轮目标",
  "objective_status": "needs_tool | waiting_user | completed | needs_human | unknown",
  "reasoning_summary": "简短推理摘要，写入日志，不直接给客户",
  "objective_state": {...},
  "objective_plan": [...],
  "plan_revision_reason": "计划为什么调整",
  "reply_to_user": "客户可见回复，needs_tool 时必须为空",
  "tool_calls": [
    {
      "name": "search_current_games",
      "arguments": {...},
      "reason": "为什么现在需要这个工具",
      "idempotency_key": null
    }
  ],
  "needs_human": false,
  "stop_reason": {
    "can_stop": false,
    "why": "为什么能停或不能停",
    "pending_work": ["还需要查候选人"],
    "depends_on_tool_results": true
  }
}
```

关键不变量

- `objective_status=needs_tool` 时必须有 `tool_calls`，且 `reply_to_user` 必须为空。
- `waiting_user/completed/needs_human/unknown` 时不能带工具调用，且必须有客户回复。
- `needs_human=true` 时状态必须是 `needs_human`。
- 契约非法时，后端不执行工具，而是把契约错误作为工具结果回喂模型修正。

这套契约的核心作用是把“模型思考”和“后端执行”拆开。模型可以提出动作，但动作必须符合结构化契约，后端才能执行。

11. 后端工具

工具注册在 `/Users/wangjie/Desktop/mahjong_agent_core/src/mahjong_agent_runtime/tools.py` 。

工具	模式	作用
<code>search_current_games</code>	<code>read_only</code>	查询当前缺人的局，并返回匹配摘要和加入后座位变化。
<code>search_customers</code>	<code>read_only</code>	根据玩法、档位、时间、烟况、性别、疲劳度、关系约束推荐候选人。
<code>create_game</code>	<code>state_write</code>	创建一个组局记录，不自动确认房间。
<code>create_invite_drafts</code>	<code>draft_write</code>	为候选人创建邀约草稿。
<code>create_outbound_message_drafts</code>	<code>draft_write</code>	创建通用外发消息草稿，支持多通道。
<code>record_candidate_reply</code>	<code>state_write</code>	记录候选人回复，如确认、拒绝、协商、到店，并更新局内座位。
<code>update_game_status</code>	<code>state_write</code>	更新局状态，如已成局、取消、归档。
<code>record_user_memory</code>	<code>state_write</code>	记录当前任务短期记忆和候选长期画像。
<code>update_context_checkpoint</code>	<code>state_write</code>	更新会话 checkpoint。
<code>record_badcase</code>	<code>audit_write</code>	记录 badcase，进入后续 eval 回归。

工具网关做什么

`ToolGateway.execute()` 是模型工具调用的唯一后端入口。它会做：

- 检查工具是否存在。
- 检查工具 schema，参数类型不合法直接拒绝。
- 检查风险级别和执行模式是否允许。
- 生成后端权威幂等键。
- 检查幂等账本，重复请求直接返回旧结果。
- 执行 handler 并记录状态变更。
- 把结果写入 trace，再回喂给模型。

12. 短期记忆与长期画像

为什么要分两层

用户在当前组局里说“我不和 C 打”，不一定代表以后永远不和 C 打。直接写长期画像会污染用户画像；完全不记又会导致本轮搜索候选人或现有局时踩雷。因此当前实现分成两层：

类型	生命周期	用途	表
TaskMemory	当前任务/当前会话立即生效	过滤当前局、过滤候选人、约束本轮回复	runtime_task_memories
PendingMemoryCandidate	候选长期事实	后续会话结束提炼、人工审核或高置信写入画像	runtime_pending_memory_candidates
CustomerProfile	长期	玩法偏好、档位偏好、烟况、响应速度、疲劳度、关系约束	runtime_customers

A 不和 C 打怎么处理

- 1. A 在当前会话里说：“我不和 C 打”。
- 2. 模型看到这是当前任务约束，调用 record_user_memory 。
- 3. 工具写入 runtime_task_memories ，字段包含 customer_id=A 、 target_customer_id=C 、 field=avoid_playing 、证据文本和置信度。
- 4. 之后模型调用 search_current_games 时，后端会过滤掉包含 C 的局。
- 5. 之后模型调用 search_customers 时，后端会排除 C。
- 6. 如果用户表达的是稳定事实，比如“以后都别给我约 C”，模型可以同时写入 pending_long_term_memories ，等待后续高置信提炼或审核。

长期画像什么时候写

当前设计方向是：用户明确表达稳定偏好时，先记录候选长期画像；会话结束后可以按 conversation_id 拉取多轮输入，交给模型提炼 JSON 契约，要求输出字段、证据、置信度和风险级别。置信度高于阈值，例如 0.9，才写入正式客户画像。

注意：长期画像写入不能只看一句话。比如“今天不和 C 打”只应是 task memory；“以后都别叫 C 跟我一桌”才更接近长期关系事实。

13. 状态机与局生命周期

局不是一来消息就创建

只有用户明确确认要组局，或者原始消息本身已经是明确组局信息，才创建 Game 。如果只是“现在有人吗”，应该先查现有局；没有合适局再问是否要组。

局模型里的关键字段

字段	说明
game_id	局 ID。
conversation_id	归属会话。
status	待补充、邀请中、已成局、已取消、已归档等。
requirement	玩法、底注、封顶、时间、人齐开、时长、烟况、人数结构等。
participants	已确认参与人。
seat_claims	联系人代表多个座位的情况，比如 lin01 代表两个人。

开始时间与结束时间

真实运营中，每个局都应该有开始时间和结束时间或过期策略。固定时间局可以由 `start_time + duration_hours` 得到结束时间；人齐开局需要设置最大等待时间，例如 4 小时内未成局自动过期归档。

短码解释

- `173` 表示当前 1 人，缺 3 人。
- `272` 表示当前 2 人，缺 2 人。
- `371` 表示当前 3 人，缺 1 人。
- `216`、`1-32`、`10-32` 这类在川麻里更接近底注/封顶表达，应拆成底注和封顶，而不是当成玩法。

14. 幂等、顺序与并发

消息级幂等

入口消息有 `message_id` 或 `source_message_id` 时，运行时会生成消息级幂等键。相同消息重复进来，直接返回之前结果，避免网络重试导致重复建局或重复发草稿。

工具级幂等

工具级幂等键由后端生成，不信任模型传来的 idempotency key：

```
conversation:{conversation_id}:sender:{sender_id}:message:{source_message_id}:tool:{tool_name}:args:{sha256(arguments)}
```

相同消息、相同工具、相同参数重复执行时，ToolGateway 会从 `runtime_idempotency_ledger` 返回旧结果。

顺序控制

- 每个 conversation 有一把锁，同一个会话内消息串行处理，避免上下文交叉。
- 每次新消息进入会推进 conversation version。
- 如果新消息来了，之前还没发出的 pending output 会被 supersede，避免用户补充信息后还发旧回复。

分布式扩展方向

当前本地 MacBook 场景使用进程内锁 + SQLite 足够。分布式部署时，需要把会话锁迁移到 Redis 分布式锁或消息队列分区顺序消费；幂等账本必须放在共享存储；同一 conversation_id 要路由到同一分区，保证消息顺序。

15. 客户可见文本审查

凡是客户可见文本都必须审查，包括最终回复、候选人邀约、外发草稿和闲聊回复。

审查什么

- 不能说“我是 AI/Agent/模型/系统”。
- 不能暴露 prompt、工具、trace、候选人评分、内部执行细节。
- 不能透露其他用户的敏感信息、老板备注、内部画像、微信 ID。
- 不能承诺未确认的房间、优惠、支付、纠纷处理。
- 不能把“我问了 8 个人”“等老板审批”这类内部流程说给客户。

为什么不是无限加 guard

理想做法是三层：

1. 主提示词约束模型，不要生成暴露内部的文本。
2. 客户可见文本审查作为 Agent loop 内部工具，发现问题后把审查结果回喂主模型重写。

3. 上线前保留少量硬 guard 作为最后兜底，只拦截高风险泄露，不承载业务语义。

16. 可观测、审计与 Eval

日志格式

日志按统一格式写入：

```
traceId=time(yyyy-mm-dd hh:mm:ss)-loglevel: content
```

主 trace 记录哪些 step

step	说明
user_input	用户输入。
context_built	上下文构建结果和预算审计。
llm_prompt	实际喂给模型的 messages。
llm_response	模型原始输出。
tool_gateway_received	模型请求调用工具。
tool_result	工具返回结果。
customer_visible_content_review_result	客户可见文本审查结果。
state_transition	状态变更。
final_output	最终回复。
context_summary_checked	是否触发摘要。

Badcase 与 Golden Dataset

当测试发现问题，不应该只补 if-else。应把案例沉淀为：

- badcase：失败样例，记录输入、期望行为、实际行为、trace。
- golden dataset：稳定的高质量样本，作为回归验证标准。
- eval：自动跑测试，评估意图、工具调用、回复质量、隐私审查是否符合预期。

17. SQLite 表设计

SQLite 初始化在 /Users/wangjie/Desktop/mahjong_agent_core/src/mahjong_agent_runtime/sqlite_store.py 的 _migrate()。

表名	主键	用途
runtime_customers	customer_id	客户长期画像，payload 存完整 JSON。
runtime_customer_relationships	pair_key	客户关系，如打过、不和谁打。
runtime_games	game_id	组局状态和需求。
runtime_invite_drafts	draft_id	候选人邀约草稿。
runtime_outbound_message_drafts	draft_id	多通道外发草稿。
runtime_state_transitions	id	状态变更审计。
runtime_conversation_turns	id	多轮对话 turn，包含 user/assistant/tool。
runtime_conversation_checkpoints	conversation_id	会话摘要 checkpoint。
runtime_conversation_versions	conversation_id	会话版本，用于过期输出控制。
runtime_idempotency_ledger	idempotency_key	工具调用幂等账本。
runtime_message_results	message_id	消息级幂等结果。
runtime_message_references	id	消息与业务对象的引用关系。
runtime_task_memories	memory_id	当前任务短期记忆。
runtime_pending_memory_candidates	candidate_id	候选长期画像。
runtime_badcases	id	badcase 归档。

为什么大量使用 payload JSON

当前阶段业务模型还在快速演进，用 JSON payload 可以减少频繁迁移表结构的成本。核心检索字段会冗余为普通列，例如 conversation_id、status、customer_id、updated_at，保证常用查询可索引。

18. Redis 与缓存定位

当前主持久化是 SQLite。Redis 更适合用于短期记忆、运行时缓存、分布式锁和热点状态，而不是作为唯一事实源。

适合放 Redis 的数据

数据	建议结构	原因
短期会话 buffer	List / Stream	按时间顺序追加消息，适合多轮拼接和延迟聚合。
当前任务锁	String + TTL	分布式环境下保证同一 conversation 串行。
幂等键	String / Hash + TTL	快速判断重复消息和重复工具调用。
人齐开等待局	Sorted Set	按过期时间扫描，自动归档超时局。
热点客户画像缓存	Hash	减少 SQLite 查询，但写入仍以 DB 为准。

如果 Redis 未启用，系统仍然可以依靠 SQLite 运行；启用 Redis 后，常客列表等数据可能来自启动时 seed、缓存同步或运行态加载，所以重启表现会有差异。生产上应明确“SQLite 为事实源，Redis 为缓存/锁/短期态”。

19. 异常与降级

异常	处理策略
模型超时	停止本轮工具执行，返回人工确认或保守回复，并写 trace。
模型输出非法 JSON	尝试解析修复；仍失败则把契约错误回喂模型或转人工。
工具参数不合法	ToolGateway 拒绝执行，把 schema 错误作为工具结果回喂模型。
文本审查不通过	把违规项和建议安全文本回喂主流程，要求重写；高风险时转人工。
重复消息	消息级幂等直接返回已有结果。
重复工具调用	工具级幂等直接返回已有工具结果。
新消息打断旧流程	conversation version 递增，未发出的旧 pending output 被 supersede。

20. 面试讲法

一句话版本

我做的是一个面向私域运营撮合场景的目标驱动 Agent。模型负责理解目标、规划工具和推进状态；后端负责工具契约、权限、幂等、状态机、审计和落库。系统已经接入 WeChaty 做微信白名单灰度测试，并沉淀了 trace、badcase、golden dataset 和 eval 回归。

如何解释上下文管理

我没有把全量历史都塞给模型，而是用 ContextBuilder 组装当前消息、近期对话、checkpoint 摘要、客户画像、客户关系、当前任务记忆、活跃局、待发送草稿、可用工具和输出契约。近期对话按 token 预算从近到远裁剪；对话变长后由摘要模型生成 checkpoint，后续上下文用 checkpoint 承接旧信息。

如何解释记忆

我把记忆拆成三层：当前任务记忆、会话 checkpoint、长期客户画像。比如用户说这局不和某人打，会立即写当前任务记忆，用于本轮搜索和候选过滤；但不会直接污染长期画像。只有稳定偏好在会话结束后通过模型提炼，带证据和置信度，超过阈值才写长期画像。

如何解释生产边界

模型不能直接改数据库，也不能直接外发消息。它只能按契约提出工具调用。后端 ToolGateway 做 schema 校验、权限校验、风险级别、幂等和状态机检查。客户可见文本还要经过审查，避免泄露系统信息和用户隐私。

21. 关键代码索引

模块	路径
HTTP 服务入口	/Users/wangjie/Desktop/mahjong_agent_core/scripts/agent_runtime_app.py
主 Agent Runtime	/Users/wangjie/Desktop/mahjong_agent_core/src/mahjong_agent_runtime/runtime.py
上下文 构建	/Users/wangjie/Desktop/mahjong_agent_core/src/mahjong_agent_runtime/context.py
工具网 关和工 具定义	/Users/wangjie/Desktop/mahjong_agent_core/src/mahjong_agent_runtime/tools.py
领域模 型	/Users/wangjie/Desktop/mahjong_agent_core/src/mahjong_agent_runtime/models.py
内存 Store	/Users/wangjie/Desktop/mahjong_agent_core/src/mahjong_agent_runtime/store.py
SQLite Store	/Users/wangjie/Desktop/mahjong_agent_core/src/mahjong_agent_runtime/sqlite_store.py
上下文 摘要	/Users/wangjie/Desktop/mahjong_agent_core/src/mahjong_agent_runtime/summary.py
主提示 词	/Users/wangjie/Desktop/mahjong_agent_core/src/mahjong_agent_runtime/prompts/agent_runtime_system.md
输入门 禁提示 词	/Users/wangjie/Desktop/mahjong_agent_core/src/mahjong_agent_runtime/prompts/wechaty_input_gate.md
测试	/Users/wangjie/Desktop/mahjong_agent_core/tests/test_agent_runtime.py